

VECTOR

Optimizarea Rutei Turistice

Holban Vlad · Nicu Eduard

Documentație tehnică — implementare C++ și aplicație web

Scopul Proiectului

Compania **Vector** este lider național în dezvoltarea de soluții software pentru industria transportului și turismului, construind platforme inteligente pentru optimizarea rutelor de livrare și planificarea itinerariilor turistice pentru agențiile partenere din România și Europa Centrală.

Proiectul vizează dezvoltarea unei aplicații pentru călătorii individuali. Un turist sosit într-un oraș necunoscut dorește să viziteze un set de locații de interes (muzee, piețe, monumente, parcuri) și să se întoarcă la hotel, **minimizând distanța totală parcursă**. Aplicația este disponibilă în două variante: o implementare **C++** cu meniu interactiv și o **aplicație web** accesibilă la <https://notvl4d.github.io/vector/>.

1. Date de Intrare / Ieșire

Parametru	Tip	Descriere
N	întreg	Numărul total de noduri (locații) din graf
M	întreg	Numărul de muchii (drumuri directe)
x, y, cost	triplet întreg	O muchie neorientată de la x la y cu costul dat
nod_start	întreg	Nodul de start — hotelul (punct de plecare și întoarcere)
nr_interese	întreg	Numărul punctelor de interes turistic
interese[]	listă întregi	Nodurile care trebuie vizitate obligatoriu

Ieșire	Descriere
Secvența de noduri	Ruta optimă: nod_start → ... → nod_start, fiecare interes vizitat o singură dată.
Cost total	Suma costurilor muchiilor parcurse (minim global găsit prin backtracking).
Mesaj de eroare	«Nu exista solutie» dacă nu există rută validă prin toate punctele de interes.

Formatul fișierului input.txt — exemplu demonstrativ

```
4 6 <- N M
1 2 10 <- x y cost (repetata de M ori)
1 3 15
1 4 20
2 3 35
2 4 25
3 4 30
```

```
1 <- nod_start  
3 <- nr_interese  
2 3 4 <- lista_puncte_de_interes
```

2. Implementare C++ — Funcție cu Funcție

Codul sursă din **src/main.cpp** este structurat în secțiuni clare, fiecare cu o singură responsabilitate. Mai jos, fiecare funcție este prezentată împreună cu codul său și o explicație concisă a rolului și logicii interne.

initializare()

```
void initializare() {  
    for(int i = 1; i <= 200; i++)  
        for(int j = 1; j <= 200; j++)  
            costuri[i][j] = 0;  
  
    cost_minim = 1000000000;  
    lungime_solutie = 0;  
}
```

Resetează starea globală înaintea oricărei citiri: pune pe 0 toată matricea de costuri `costuri[200][200]`, inițializează `cost_minim` la o valoare foarte mare (1 miliard, echivalent practic cu infinit) și `lungime_solutie` la 0, indicând că nu există încă nicio soluție salvată.

citire_din_fisier()

```
void citire_din_fisier() {
    initializare();

    citit_din_fisier = true;

    fin >> noduri >> muchii;

    for(int i = 1; i <= muchii; i++) {
        fin >> x >> y >> cost;

        costuri[x][y] = costuri[y][x] = cost; // graf neorientat
    }

    fin >> nod_start >> nr_interese;

    for(int i = 1; i <= nr_interese; i++)
        fin >> interese[i];
}
```

Citește datele din fișierul input.txt prin stream-ul global fin. Construiește matricea de adiacență simetrică ($\text{costuri}[x][y] = \text{costuri}[y][x] = \text{cost}$) deoarece graful este neorientat. Setează fanionul `citit_din_fisier = true`, folosit ulterior de `afisare_rezultat()` pentru a decide dacă outputul merge în fișier sau pe ecran.

citirea_de_la_tastatura()

```
void citirea_de_la_tastatura() {
    initializare();

    citit_din_fisier = false;

    cin >> noduri >> muchii;

    for(int i = 1; i <= muchii; i++) {
        cin >> x >> y >> cost;
        costuri[x][y] = costuri[y][x] = cost;
    }

    cin >> nod_start >> nr_interese;

    for(int i = 1; i <= nr_interese; i++)
        cin >> interese[i];
}
```

Funcție identică ca structură cu citire_din_fisier(), dar folosește cin în loc de fin. Pune citit_din_fisier = false, astfel încât rezultatul final va fi afișat pe ecran (cout) în loc să fie scris în output.txt.

OK(int k)

```
bool OK(int k) {
    // Verificare muche valida intre nodul anterior si cel curent
    if(k > 1 && costuri[vizitat[k-1]][vizitat[k]] == 0)
        return false;

    // Verificare unicitate: nodul nu a mai fost vizitat
    for(int i = 1; i < k; i++)
        if(vizitat[i] == vizitat[k])
            return false;

    return true;
}
```

Funcția de validare a unui candidat în backtracking. Realizează două verificări: (1) Există muchia dintre nodul anterior vizitat[k-1] și cel curent vizitat[k] — adică costuri[vizitat[k-1]][vizitat[k]] != 0; (2) Nodul curent nu se află deja în stiva vizitat[1..k-1], garantând că fiecare locație este vizitată cel mult o dată. Dacă oricare condiție eșuează, candidatul este respins.

Solutie(int k)

```
bool Solutie(int k) {
    cost_solutie = 0;

    bool freq[201] = {0};

    // Marcheaza toate punctele de interes in freq[]
    for(int i = 1; i <= nr_interese; i++)
        freq[interese[i]] = 1;

    // Parcurge stiva: bifeaza interesele intalnite si calculeaza costul
    for(int i = 1; i <= k; i++) {
        if(freq[vizitat[i]] == 1) freq[vizitat[i]] = 0;
        if(i > 1) cost_solutie += costuri[vizitat[i-1]][vizitat[i]];
    }

    // Verifica daca mai exista interese nevizitate
    for(int i = 1; i <= 200; i++)
        if(freq[i] == 1) return false;

    // Verifica drum de intoarcere la hotel
    if(costuri[vizitat[k]][vizitat[1]] == 0) return false;

    cost_solutie += costuri[vizitat[k]][vizitat[1]];

    // Accepta doar daca e mai buna decat minimul curent
    if(cost_solutie < cost_minim) {
        cost_minim = cost_solutie;
        return true;
    }

    return false;
}
```

Verifică dacă stiva curentă vizitat[1..k] reprezintă o soluție completă și optimă. Folosește un vector de frecvență freq[] pentru a bifa punctele de interes pe măsură ce sunt întâlnite în stivă. O soluție este validă dacă: (a) toate interesele sunt bifate (freq[] golit), (b) există muchie de întoarcere de la ultimul nod la nod_start, și (c) costul total este strict mai mic decât cost_minim curent. Actualizează cost_minim și returnează true doar dacă toate trei condiții sunt îndeplinite.

Salvare(int k)

```
void Salvare(int k) {  
    for(int i = 1; i <= k; i++)  
        solutie[i] = vizitat[i];  
    lungime_solutie = k;  
}
```

Copiază conținutul stivei vizitat[1..k] în array-ul solutie[], reținând ruta optimă găsită până în acel moment. Actualizează lungime_solutie = k, astfel încât afisare_rezultat() să știe câte noduri să afișeze. Este apelată din Back() imediat după ce Solutie() returnează true.

Back(int k)

```
void Back(int k) {  
    for(int i = 1; i <= noduri; i++) {  
        vizitat[k] = i; // plaseaza candidatul i pe pozitia k  
  
        if(OK(k)) { // candidat valid?  
            if(Solutie(k)) // solutie completa si mai buna?  
                Salvare(k); // -> salveaza  
            else  
                Back(k + 1); // -> extinde recursiv  
        }  
    }  
}
```

Nucleul algoritmului de backtracking. Pentru fiecare poziție k din stivă, încearcă pe rând toți cei N candidați posibili (nodurile 1..noduri). Un candidat este plasat provizoriu în vizitat[k] și validat prin OK(k). Dacă este valid, se verifică dacă stiva curentă formează deja o soluție completă (Solutie(k)); dacă da, se salvează, altfel se extinde recursiv la poziția k+1. Apelul inițial este Back(2), deoarece vizitat[1] = nod_start este fixat în main().

afisare_rezultat()

```
void afisare_rezultat() {
    if(lungime_solutie == 0) {
        if(citit_din_fisier) fout << "Nu exista solutie\n";
        else cout << "Nu exista solutie\n";
        return;
    }

    if(citit_din_fisier) {
        for(int i = 1; i <= lungime_solutie; i++)
            fout << solutie[i] << ' ';
        fout << solutie[1] << '\n'; // inchide circuitul
        fout << "Cost: " << cost_minim << '\n';
        cout << "Rezultatul a fost salvat in output.txt\n";
    } else {
        for(int i = 1; i <= lungime_solutie; i++)
            cout << solutie[i] << ' ';
        cout << solutie[1] << '\n';
        cout << "Cost: " << cost_minim << '\n';
    }
}
```

Afișează ruta optimă salvată în `solutie[]`. Dacă `lungime_solutie == 0`, algoritmul nu a găsit nicio soluție și se afișează mesajul corespunzător. Altfel, se tipăresc toate nodurile din `solutie[1..lungime_solutie]` urmate de `solutie[1]` pentru a închide circuitul (revenire la hotel), și costul minim. Destinația (fișier sau ecran) este aleasă în funcție de fanionul `citit_din_fisier`.

main()

```
int main() {  
  
    int menu;  
  
    do {  
  
        // Afiseaza meniul si citeste optiunea  
  
        cout << "1. Citire din fisier\n";  
  
        cout << "2. Citire de la tastatura\n";  
  
        cout << "3. Rezolvare\n";  
  
        cout << "4. Afisare rezultat\n";  
  
        cout << "5. Iesire\n";  
  
        cin >> menu;  
  
        switch(menu) {  
  
            case 1: citire_din_fisier(); break;  
            case 2: citirea_de_la_tastatura(); break;  
            case 3:  
                if(nod_start == 0) { /* eroare */ break; }  
                vizitat[1] = nod_start;  
  
                Back(2); // lanseaza backtracking-ul  
  
                break;  
            case 4: afisare_rezultat(); break;  
            case 5: break;  
        }  
  
    } while(menu != 5);  
  
    fin.close(); fout.close();  
  
    return 0;  
}
```

Punctul de intrare al programului. Implementează o buclă do-while cu un meniu interactiv de 5 opțiuni: (1) citire din fișier, (2) citire de la tastatură, (3) rezolvare — fixează `vizitat[1] = nod_start` și lansează `Back(2)`, (4) afișare rezultat, (5) ieșire. Opțiunea 3 verifică mai întâi dacă datele au fost citite (`nod_start != 0`) înainte de a porni algoritmul, prevenind o rulare cu date neinițializate. La final, închide stream-urile.

3. Exemplu de Rulare

Graful demonstrativ conține 4 noduri și 6 muchii (graf complet K_4). Nodul de start este 1; punctele de interes sunt 2, 3 și 4.

Rută candidată	Cost	Acceptată?
1 → 2 → 3 → 4 → 1	10+35+30+20 = 95	Da — prima soluție, cost_minim = 95
1 → 2 → 4 → 3 → 1	10+25+30+15 = 80	Da — îmbunătățire, cost_minim = 80
1 → 3 → 2 → 4 → 1	15+35+25+20 = 95	Nu — cost $\geq$ cost_minim
1 → 3 → 4 → 2 → 1	15+30+25+10 = 80	Nu — cost egal, nu strict mai mic
1 → 4 → 2 → 3 → 1	20+25+35+15 = 95	Nu — cost $\geq$ cost_minim
1 → 4 → 3 → 2 → 1	20+30+35+10 = 95	Nu — cost $\geq$ cost_minim

output.txt — rezultat final

```
1 2 4 3 1
```

```
Cost: 80
```

4. Complexitate și Limitări

Aspect	Detaliu
Complexitate timp	$O(N!)$ în cazul defavorabil — se explorează toate permutările
Complexitate spațiu	$O(N^2)$ — matricea costuri[200][200] alocată static
Domeniu optim	$N \leq 15$ noduri pentru timp de execuție rezonabil (sub 1 secundă)
Limită matrice	Maxim 200 noduri (declarație statică); indecși de la 1 la N
Tip graf acceptat	Neorientat, ponderat, parțial sau complet; 0 în matrice = lipsă muchie